



Ratehopper3

SECURITY AUDIT REPORT

———— 11 June 2026 ————

Prepared by: **Shred Security**

Security Researchers
kenzo, yashar

Table of Contents

- [Table of Contents](#)
- [About Shred Security](#)
- [Protocol Executive Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Executive Summary](#)
- [Findings](#)

About Shred Security

Shred Security is a research-driven security firm specializing in smart contract auditing, threat modeling, and blockchain security. Our audits consistently uncover high-impact vulnerabilities missed by others, backed by a proven track record of top public competition placements and bounty hunting, security partnerships with leading protocols.

Learn more about us: shredsec.xyz

Protocol Executive Summary

Ratehopper is an intelligent DeFi platform that helps you find the best lending and borrowing rates across multiple protocols and automate position switching to maximize your returns. In addition, we have different strategies that allow you to deploy different investing strategies based on your risk appetite.

The team introduced a Gnosis Safe module for Uniswap V3 WETH/USDC LP position management (PR #12) via `RatehopperUniV3Positions.sol`, exposing `openLp`, `collectLp`, and `closeLp` as the full LP-leg lifecycle while leaving lending supply, borrow, and debt repayment to separate Safe transactions. This update includes USDC-to-WETH swapping through pinned SwapRouter02, LP NFT minting and module-mediated close flows (fee harvest, partial or full liquidity removal via `exitBps`, principal collection, burn, and WETH-to-USDC swap-back), mid-position fee collection with configurable treasury skim (`feeCollectBps`) and performance fees on net profits, integration with `ProtocolRegistry`, AccessControl and timelock-gated critical configuration, user-supplied mint and decrease-liquidity minimums with a 3% maximum slippage cap, minimum pool liquidity enforcement, token and misdirected NFT rescue paths, canonical Base Uniswap V3 address configuration, shared TimelockController deployment, and expanded integration tests for LP-backed leveraged positions.

Disclaimer

A smart contract security review cannot guarantee the complete elimination of vulnerabilities. The process is limited by time, available resources, and human expertise, and is intended to identify as many potential issues as reasonably possible. As such, no assurance can be given that all vulnerabilities will be discovered, or that the reviewed smart contracts are entirely secure. To strengthen security over time, follow-up audits, bug bounty programs, and continuous on-chain monitoring are strongly recommended.

Risk Classification

Likelihood \ Impact	High	Medium	Low
High	High	High/Medium	Medium
Medium	High/Medium	Medium	Medium/Low
Low	Medium	Medium/Low	Low

Executive Summary

The shred security team has conducted the review for 3 days in total. In this period of time, a total of 8 issues were found: 5 low-severity and 3 informational. No medium- or high-severity issues were identified.

About the Project

Project Name	Ratehopper Contracts
Repository	https://github.com/RateHopper/ratehopper-contracts
Contracts	`RatehopperUniV3Positions.sol`
Commit/PR	1801145 / PR #12
Type of Project	Intelligent DeFi platform
Lines of Code	400

Audit Timeline

Audit Start	30/05/2026
Audit End	03/06/2026
Report Published	11/06/2026

Vulnerability Summary

Severity	Count	Fixed	Acknowledged
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	5	5	0
Informational	3	1	2
Total	8	6	2

Findings Summary

Issue ID	Description	Severity	Status
L-1	Missing position validation allows operator to manage unregistered WETH/USDC LP positions	Low	Fixed
L-2	No minimum minted position liquidity	Low	Fixed
L-3	Blacklisted treasury bricks `closeLp()` / `collectLp()` (exit DoS)	Low	Fixed
L-4	Partial close reduces basis without removing liquidity	Low	Fixed
L-5	`slippageBps` validated but never enforced on swaps	Low	Fixed
I-1	Safe-owned Uniswap V3 positions allow direct fee collection outside the protocol fee flow	Informational	Acknowledged
I-2	`uint128` for residual basis and fee amounts	Informational	Acknowledged
I-3	`_validatePool()` does not verify `token0` / `token1`	Informational	Fixed

Findings

Low Severity

[L-1] Missing position validation allows operator to manage unregistered WETH/USDC LP positions

Severity: Low

Affected: `collectLp()`, `_collectLp()`

Summary

`collectLp()` accepts any WETH/USDC NPM position owned by the Safe. It does not require the NFT to have been opened and registered through `openLp()`.

Technical Details

`closeLp()` requires a stored basis:

```
uint128 residualBasis = residualBasisUsd6Of[tokenId];  
if (residualBasis == 0) revert UnknownPosition();
```

`collectLp()` only checks token pair and ownership:

```
_requireWethUsdcPositionOwnedBy(_onBehalfOf, tokenId);  
_collectLp(_onBehalfOf, tokenId);
```

Impact

The Safe or `registry.safeOperator()` can route unrelated Safe-owned WETH/USDC LP NFTs through RHP and charge `feeCollectBps` on their fees. This expands operator authority beyond protocol-managed positions.

Recommendation

Require an active RHP position before fee collection:

```

        function collectLp(address _onBehalfOf, uint256 tokenId)
external ... {
+         if (residualBasisUsd6Of[tokenId] == 0) revert
UnknownPosition();
        _requireWethUsdcPositionOwnedBy(_onBehalfOf, tokenId);
        _collectLp(_onBehalfOf, tokenId);
    }

```

[L-2] No minimum minted position liquidity

Severity: Low

Affected: `openLp()`, `_safeMintLp()`

Summary

`openLp()` validates pool depth through `_validatePool()`, but it does not enforce a minimum `liquidity` value returned by NPM `mint()`.

Technical Details

`_safeMintLp()` discards the returned liquidity:

```

(tokenId, , amount0Out, amount1Out) =
    abi.decode(ret, (uint256, uint128, uint256, uint256));

```

Impact

Very low-liquidity NFTs make `liquidityToRemove == 0` easier to hit in L-4. This is a hardening issue and widener for the basis/liquidity desync, not an independent fund-loss path for normal-sized positions.

Recommendation

Decode and enforce a minimum minted liquidity:

```

-         (tokenId, , amount0Out, amount1Out) =
+         (tokenId, liquidityMinted, amount0Out, amount1Out) =
            abi.decode(ret, (uint256, uint128, uint256, uint256));

```

```
+     if (liquidityMinted < minPositionLiquidity) revert  
PositionLiquidityTooLow();
```

[L-3] Blacklisted treasury bricks `closeLp()` / `collectLp()` (exit DoS)

Severity: Low

Affected: `closeLp()`, `collectLp()`, `_collectLp()`, `_chargeCollectFee()`

Summary

`_chargeCollectFee()` sends the collect fee to `treasury` with a hard `safeTransfer`. If `treasury` is USDC-blacklisted, any collection with a non-zero USDC treasury cut reverts.

Technical Details

The performance-fee path in `closeLp()` is non-fatal on transfer failure, but the collect-fee path is not:

```
fee = (amount * feeCollectBps) / 10_000;  
if (fee > 0) IERC20(token).safeTransfer(treasury, fee);
```

`closeLp()` calls `_collectLp()` before decreasing liquidity, so this can block an affected close before principal removal.

Impact

Affected `collectLp()` and `closeLp()` calls revert until `treasury` or `feeCollectBps` is changed through the timelocked admin path. The issue requires `feeCollectBps > 0`, a non-zero USDC fee leg, and a blacklisted treasury.

Recommendation

Make collect-fee transfer failure non-fatal and forward the full amount to the Safe when the treasury transfer fails:

```
-     if (fee > 0) IERC20(token).safeTransfer(treasury, fee);  
-     uint256 toSafe = amount - fee;  
+     uint256 toSafe = amount;  
+     if (fee > 0) {
```

```
+         try IERC20(token).transfer(treasury, fee) returns (bool
ok) {
+             if (ok) toSafe = amount - fee;
+             else fee = 0;
+         } catch {
+             fee = 0;
+         }
+     }
    if (toSafe > 0) IERC20(token).safeTransfer(_onBehalfOf,
toSafe);
```

[L-4] Partial close reduces basis without removing liquidity

Severity: Low

Affected: `closeLp()`

Summary

Partial `closeLp()` reduces `residualBasisUsd6Of[tokenId]` before computing the NPM liquidity to remove. Because both values are rounded down independently, basis can be reduced while `liquidityToRemove == 0` for very low-liquidity positions.

Technical Details

Basis is updated before liquidity removal:

```
basisForExit = Math.mulDiv(residualBasis, exitBps,
10_000).toUint128();
residualBasisUsd6Of[tokenId] = residualBasis - basisForExit;

liquidityToRemove = Math.mulDiv(liquidity, exitBps,
10_000).toUint128();
if (liquidityToRemove > 0) {
    // decreaseLiquidity + collect principal
}
```

The bug occurs when `liquidity * exitBps < 10_000` but `residualBasis * exitBps >= 10_000`. Since `exitBps` can be as low as `1`, the clearest trigger is a dust or edge-case position with `liquidity < 10_000`. Normal user-sized WETH/USDC LPs are expected to have much larger NPM liquidity.

Impact

For dust or pathological low-liquidity positions, stored basis can be lowered while the LP remains unchanged. A later close measures profit against an artificially low basis, causing the Safe to pay more `performanceFeeBps` than intended. Access is limited to the Safe or `registry.safeOperator()`.

Recommendation

Revert when basis would be reduced but no liquidity is removed:

```
uint128 liquidityToRemove = exitBps == 10_000
    ? liquidity
    : Math.mulDiv(uint256(liquidity), uint256(exitBps),
10_000).toUint128();

+     if (exitBps != 10_000 && basisForExit > 0 &&
liquidityToRemove == 0) {
+         revert InvalidExitBps();
+     }
```

[L-5] `slippageBps` validated but never enforced on swaps

Severity: Low

Affected: `openLp()`, `closeLp()`

Summary

`openLp()` and `closeLp()` validate `slippageBps`, but never use it after the entry checks. Swap protection is enforced only through caller-supplied `swapAmountOutMin`.

Technical Details

Entry checks:

```
if (slippageBps == 0) revert SlippageTooLow();
if (slippageBps > maxSlippageBps) revert SlippageAboveMax();
```

Swap calldata uses only `swapAmountOutMin`:

```
ExactInputSingleParams({
    amountIn: uint256(wethToSwap), // or halfUsdc in openLp
```

```
    amountOutMinimum: swapAmountOutMin,  
    sqrtPriceLimitX96: 0  
  })
```

`minUsdcOut` and `decreaseAmount*Min` are separate `closeLp()` guards and do not implement `slippageBps`.

Impact

An operator or client can pass a tight-looking `slippageBps` with a weak `swapAmountOutMin` and still pass all checks. On `closeLp()`, this can affect the full WETH unwind leg. Access is limited to the Safe or `registry.safeOperator()`.

Recommendation

Keep `slippageBps` and `maxSlippageBps`; add quoter-derived `expected*` amounts on `openLp` / `closeLp` and, before any swap or NPM call, require each supplied minimum to be at least $(\text{expected} * (10_000 - \text{slippageBps})) / 10_000$ for that leg. Reject `0` mins in production paths.

Informational

[I-1] Safe-owned Uniswap V3 positions allow direct fee collection outside the protocol fee flow

Severity: Informational

Affected: `openLp()`, `_safeMintLp()`, `collectLp()`, `_collectLp()`

Summary

LP NFTs are minted to the Safe, not held by RHP. The Safe owner can call Uniswap V3 `NonfungiblePositionManager.collect()` directly and receive accrued fees without going through `collectLp()`, so `feeCollectBps` is never applied on that harvest path.

Technical Details

Mint sends the NFT to the Safe:

```
recipient: _onBehalfOf,
```

Protocol-managed harvest charges fees only when RHP collects module-mediated:

```
function collectLp(...) external onlyOperatorOrSafe(_onBehalfOf) {
    _collectLp(_onBehalfOf, tokenId);
}

// inside _collectLp:
_collectToRecipient(_onBehalfOf, tokenId, address(this), step);
_chargeCollectFee(token0, collected0, _onBehalfOf);
_chargeCollectFee(token1, collected1, _onBehalfOf);
```

A Safe transaction that calls NPM `collect()` with `recipient = Safe` clears `tokensOwed` without touching RHP.

Impact

The Safe owner can intentionally bypass `feeCollectBps` on fee harvests. User funds are not at risk and no third party can steal fees; protocol fee collection depends on using `collectLp()` / `closeLp()` rather than direct NPM calls. `closeLp()` still harvests

remaining fees through RHP before unwind, so bypass only affects fees collected outside RHP before close.

Recommendation

Document that fee harvests must use `collectLp()` or `closeLp()`. If on-chain enforcement is required, hold the NFT in a protocol-controlled contract or wrapper and route all NPM actions through RHP.

[I-2] `uint128` for residual basis and fee amounts

Severity: Informational

Affected: `openLp()`, `closeLp()`, `collectLp()`, storage and events

Summary

Several balance deltas and accounting fields are narrowed to `uint128` via `toUint128()`, including `residualBasisUsd6Of`, `currentValueUsd6`, `feeUsd6`, `usedUsdc`, and NPM-derived amounts. NPM itself uses `uint128` for liquidity and `tokensOwed`, but ERC-20 balances and mint/collect outputs are `uint256`.

Technical Details

Examples:

```
mapping(uint256 tokenId => uint128 residualBasisUsd6) public
residualBasisUsd6Of;

uint128 currentValueUsd6 = (USDC.balanceOf(_onBehalfOf) -
usdcBefore).toUint128();
uint128 usedUsdc;
amount1Used = amount1Out.toUint128();
```

`type(uint128).max` is $\sim 3.4 \times 10^{38}$ raw units. For USDC with 6 decimals that is far above realistic position sizes on Base, so truncation is not reachable under expected deployment limits.

Impact

No demonstrated exploit at realistic WETH/USDC position sizes. The pattern adds a latent footgun if future pools, tokens, or position sizes exceed `uint128` while intermediate math stays in `uint256` (silent truncation on cast). `usedUsdc` as `uint128` is especially mismatched because USDC deposit amounts are naturally `uint256`.

Recommendation

Use `uint256` for balance deltas and fee math; cast to `uint128` only where NPM or storage layout requires it, with explicit bounds checks. Alternatively document maximum position and basis invariants in NatSpec and revert when `toUint128()` would truncate.

[I-3] `_validatePool()` does not verify `token0` / `token1`

Severity: Informational

Affected: `_validatePool()`, `openLp()`, `closeLp()`, `_collectLp()`

Summary

`_validatePool()` checks pool existence, initialization, and optional liquidity floor, but not `token0 == WETH` and `token1 == USDC`.

Technical Details

```
function _validatePool(address pool) internal view returns (uint160
sqrtPriceX96) {
    if (pool == address(0)) revert PoolDoesNotExist();
    (sqrtPriceX96, , , , , ) = IUniswapV3Pool(pool).slot0();
    if (sqrtPriceX96 == 0) revert PoolNotInitialized();
    if (floor > 0 && IUniswapV3Pool(pool).liquidity() < floor)
revert PoolTooThin();
}
```

The constructor enforces `WETH < USDC`, `_safeMintLp()` hardcodes WETH/USDC, and Uniswap V3 pools sort tokens by address.

Impact

No practical exploit on canonical Uniswap V3 pools. Adding the check improves local invariants and deployment safety.

Recommendation

Add the pair assertion inside `_validatePool()`:

```
function _validatePool(address pool) internal view returns
(uint160 sqrtPriceX96) {
    if (pool == address(0)) revert PoolDoesNotExist();
+     if (IUniswapV3Pool(pool).token0() != address(WETH) ||
```

```
+         IUniswapV3Pool(pool).token1() != address(USDC)) revert  
WrongTokenPair();  
    (sqrtPriceX96, , , , , ) = IUniswapV3Pool(pool).slot0();
```